Stock Trading App

Problem Statement

Background

A company runs a simulated stock trading platform where users hold funds, buy and sell shares and track their portfolio. Stock prices are dummy values, and there is no connection to a real market. You are required to design and build a system that manages users and funds, stocks, orders and trades, holdings and charges, and portfolio and alerts. The system must follow the business rules described in each module below.

Module 1 — User, Account and Funds

- A user registers with a name, a unique PAN number, an email and a phone number. A trading account is created for the user, and the user can trade only after an Admin approves the KYC.
- A user can add funds from a linked bank account. The amount must be at least Rs. 100 and at most Rs. 1,00,000 per transaction.
- Funds are split into Available and Blocked. When a buy order is placed, the required amount is blocked. It is released if the order is cancelled.
- A user can withdraw only the Available funds. The balance can never be negative.
- A user can keep up to 20 stocks in a watchlist.

Module 2 — Stocks and Market

- Only an Admin can list stocks. A stock has a unique symbol, a company name, a sector (IT, Banking, Pharma, Energy, or Auto), a current price greater than zero and a status (Active or Suspended).
- Prices are simulated. An Admin can update the price of a stock. A price cannot change by more than 10% from the previous close in one day.
- A Suspended stock cannot be traded. Orders for it are rejected, but existing holdings stay visible.
- The market is open from 9:15 AM to 3:30 PM, Monday to Friday. Orders placed outside market hours are rejected.

Module 3 — Order Placement

- An order has a side (Buy or Sell) and a type (Market or Limit). The quantity must be a whole number from 1 to 1,000.
- A Market order is executed immediately at the current price.
- A Limit order has a price. A Buy limit order executes only when the market price is equal to or below the limit price. A Sell limit order executes only when the market price is equal to or above the limit price. Until then, it stays Pending.
- A Buy order needs enough Available funds for quantity x price plus charges. A Sell order needs enough shares in holdings that are not already part of another Pending sell order.
- Order status is Pending, Executed, Cancelled or Rejected. A user can cancel only a Pending order.
- A Pending Limit order that is not executed by market close expires and is Cancelled.
- A user can have at most 25 Pending orders at a time.

Module 4 — Trades, Holdings and Charges

- Every execution creates a Trade record with a trade ID, the order ID, the price, the quantity and the time. An Order is the intent to trade, and a Trade is the actual execution.
- After a Buy trade, the holding quantity increases and the average buy price is recalculated as a weighted average. After a Sell trade, the holding quantity decreases. A holding with zero quantity is removed.
- Short selling is not allowed. A user cannot sell shares they do not own.
- Brokerage is 0.05% of the trade value, capped at Rs. 20 per trade, plus 18% GST on the brokerage. It is deducted from the Available funds.
- Realized profit or loss on a Sell trade = (sell price - average buy price) x quantity.
- Unrealized profit or loss of a holding = (current price - average buy price) x quantity.

Module 5 — Portfolio and Alerts

- The portfolio shows all holdings, the invested value, the current value, the overall profit or loss and the percentage return.
- A user can set a price alert for a stock with a target price. The alert triggers once when the price crosses the target. A user can have at most 10 active alerts.
- A daily statement lists all orders, trades and charges of the day.
- An Admin can Block an account. A Blocked account cannot place Buy orders, but it can still place Sell orders.

SQL Exercise

Based on the system described above, write SQL queries for the following:

Query 1

For each sector, find the total number of stocks listed, the total number of executed trades, and the total traded value. Show only sectors that have more than 2 stocks. Sort by total traded value in descending order.

Query 2

For each user, list the total number of orders placed, the number of executed orders, and the total brokerage paid. Group the results by user and include only users who have placed at least 3 orders.

Query 3

For each stock, find the total quantity bought, the total quantity sold, and the number of distinct users who traded it. Include only stocks traded by more than 2 distinct users, ordered by total traded quantity in descending order.

Instructions

Diagrams

Draw an Entity Relationship diagram showing all the database tables you would create and the relationships between them (one-to-many, many-to-many, etc.). Also draw a high-level system diagram showing how the modules interact with each other.

Application

Build a console application. You do not need to connect to any database.

Data Storage

Use in-memory collections (such as HashMap, List, or equivalent in your language) to store and retrieve all records at runtime. You can hardcode a few users with some funds, and stocks with sectors and starting prices as initial data to start the program. All other data — orders, trades, holdings, watchlists, price alerts, charges — must be handled dynamically: add to the collection when created, update in place, and remove when deleted or cancelled.

OOP Concepts

Your code must clearly demonstrate the following object-oriented concepts:

- **Encapsulation** — keep fields private and expose them through methods.
- **Inheritance** — identify entities that share common properties and model them using a base class (for example, Customer and Admin can share a User base class, and MarketOrder and LimitOrder can extend an Order base class).
- **Polymorphism** — handle different behaviors through a common type (for example, a MarketOrder and a LimitOrder each having its own rule for when it executes).
- **Abstraction** — hide details behind an abstract class or interface (for example, an Order abstract class with a canExecute() method, or a ChargeCalculator interface).